Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

Code

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# ------------------------------
# Load Iris Dataset
# ------------------------------
iris = load_iris()

X = iris.data
y = iris.target

# Create DataFrame (Optional)
df = pd.DataFrame(X, columns=iris.feature_names)
df['Target'] = y

print("First 5 Records:\n")
print(df.head())

# ------------------------------
# Split Dataset
```

```
# ----------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# ----------------------------
# Feature Scaling
# ----------------------------
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ----------------------------
# Artificial Neural Network
# (Backpropagation)
# ----------------------------
ann = MLPClassifier(
    hidden_layer_sizes=(10,),
    activation='logistic',
    solver='sgd',
    learning_rate_init=0.1,
    max_iter=1000,
    random_state=42
)

ann.fit(X_train, y_train)

# ----------------------------
```

```
# Prediction
# ----------------------------
y_pred = ann.predict(X_test)

# ----------------------------
# Results
# ----------------------------
print("\nActual Labels:")
print(y_test)

print("\nPredicted Labels:")
print(y_pred)

print("\nAccuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=iris.target_names))
# ----------------------------
# Test New Sample
# ----------------------------
new_sample = [[5.1, 3.5, 1.4, 0.2]]
new_sample = scaler.transform(new_sample)
prediction = ann.predict(new_sample)
print("\nNew Sample Prediction:")
print("Predicted Class:", iris.target_names[prediction[0]])
```

```
First 5 Records:

   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   Target
0       0
1       0
2       0
3       0
4       0

Actual Labels:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Predicted Labels:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Accuracy: 1.0

Confusion Matrix:
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        19
  versicolor       1.00      1.00      1.00        13
   virginica       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45


New Sample Prediction:
Predicted Class: setosa
```